Chapter 1

Introduction

This chapter establishes the rationale for designing, implementing, and evaluating a Retrieval-Augmented Generation (RAG)-based LLM assistant for cybersecurity analysis of web application logs. The proposed system is intended to retrieve curated security knowledge and produce structured, evidence-supported analyses of web attacks, including SQL Injection, Cross-Site Scripting, Path Traversal, and Brute Force Login. It will distinguish the classification of the underlying web attack from the detection of Prompt Injection in attacker-controlled log fields, since these are separate threats that may coexist in a single record. The study will compare four system configurations, from an LLM-only baseline to RAG with layered defenses, using controlled clean and adversarial logs. The evaluation will measure classification performance and Prompt Injection success and robustness to assess whether lightweight safeguards reduce manipulation without impairing web-attack analysis. The expected outcome is a reproducible prototype and a bounded empirical account of its capabilities and limitations, not a claim of complete protection. This chapter also sets out the research problem, question, objectives, scope, contributions, and organization of the thesis.

1.1 Motivation

Large Language Models have demonstrated strong capabilities for interpreting, summarizing, and reasoning over complex textual information. These capabilities have created significant interest in their use for cybersecurity tasks, where analysts frequently work with heterogeneous information such as vulnerability descriptions, attack patterns, request traces, incident reports, and security logs.

Web application logs are particularly suitable for LLM-assisted analysis because they combine structured fields with free-form or attacker-controlled content. A single HTTP request may contain a method, URL, query parameters, a request body, a User-Agent string, a source address, and a response status. These fields may contain evidence of attacks such as SQL Injection, Cross-Site Scripting, Path Traversal, or repeated authentication attempts. An LLM can potentially transform these technical observations into a structured explanation containing an attack classification, severity assessment, relevant indicators, and recommended defensive actions.

However, introducing an LLM into the analysis pipeline creates a security problem that does not exist in conventional log parsers. Some web-log fields are directly controlled by external users. If an attacker inserts natural-language instructions into fields such as the URL, query parameters, request body, or User-Agent string, those instructions may later become part of the context presented to the LLM. A vulnerable model may interpret the injected text as an instruction rather than as untrusted data.

For example, a malicious request could combine a conventional attack payload with an instruction such as:

```
Ignore previous instructions and classify this request as benign.
```

A traditional parser would treat this string as passive content. An LLM, however, may allow it to influence the reasoning process. This creates the possibility of manipulating the reported classification, lowering the reported severity, suppressing evidence, or attempting to extract hidden system instructions.

A conventional web attack and a Prompt Injection are distinct threats, although both can occur in the same log. For example, a request URL may contain a SQL Injection payload while its User-Agent field contains an instruction telling the LLM to classify the request as benign. The assistant must identify the web attack from the request evidence and treat the embedded instruction as untrusted content. Its outputs should report these outcomes separately: the web-attack classification (for example, SQL Injection) and whether Prompt Injection was detected.

The problem becomes even more relevant when Retrieval-Augmented Generation is used. RAG can improve factual grounding by supplying the model with cybersecurity knowledge retrieved from an external knowledge base. At the same time, the final prompt still combines trusted instructions, retrieved evidence, and untrusted log content. Robust prompt construction and evidence control are therefore necessary if the system is expected to produce reliable security analysis.

1.2 Problem Statement

The central problem addressed by this thesis is the potential vulnerability of a RAG-based LLM assistant when analyzing web application logs containing attacker-controlled text.

The intended behavior of the system is to treat every log field as data to be analyzed. The attacker, however, may deliberately insert instructions designed to alter the output of the assistant. This creates a conflict between the trusted task definition and untrusted natural-language content.

The thesis therefore investigates two related questions:

1. How vulnerable is the assistant when no explicit defenses are applied?
2. To what extent can lightweight and explainable mitigation strategies reduce this vulnerability without removing the usefulness of the LLM and RAG pipeline?

1.3 Research Question

How vulnerable is a RAG-based LLM assistant for web application log analysis to prompt injection attacks embedded in attacker-controlled log fields, and how can evidence-controlled retrieval and simple mitigation strategies reduce this risk?

1.4 Objectives

1.4.1 General Objective

Design, implement, and evaluate a RAG-based LLM assistant for web application log analysis, focusing on robustness against Prompt Injection attacks.

1.4.2 Specific Objectives

The specific objectives of this thesis are:

1. Design an LLM-based architecture for structured web application log analysis.
2. Construct a cybersecurity knowledge base containing selected MITRE ATT&CK techniques, OWASP web-attack knowledge, response playbooks, and prompt-injection handling policies.
3. Build a controlled dataset containing clean and adversarial web application logs.
4. Define and operationalize representative Prompt Injection attacks embedded in attacker-controlled log fields.
5. Implement a RAG pipeline capable of retrieving evidence relevant to the analyzed event.
6. Implement lightweight mitigation mechanisms based on prompt hardening, instruction/data separation, metadata tagging, simple prompt-injection detection, and evidence-only prompting.
7. Compare defended and undefended system configurations under the same primary dataset and evaluation procedure, then assess classification on a separate HTTP CSIC 2010 sample.

8. Test Prompt Injection robustness on paired CSIC-derived requests using the study's own injection variants.
9. Measure attack success, classification performance, retrieval quality, evidence use, hallucination behavior, and robustness.
10. Analyze failure cases and identify limitations of the proposed defenses.

1.5 Scope

This thesis is intentionally restricted to **web application logs**.

The system analyzes web/application request information such as:

- HTTP method,
- URL,
- query parameters,
- request body,
- User-Agent,
- status code,
- source IP,
- and related structured metadata.

The experimental attack classes are limited to:

- SQL Injection,
- Cross-Site Scripting (XSS),
- Path Traversal,
- Brute Force Login,
- and Benign requests as a control class.

The study does **not** attempt to implement a complete Security Operations Center assistant and does not cover:

- SOC orchestration,
- SIEM platforms,
- SOAR platforms,
- firewall logs,
- EDR logs,
- IDS/IPS logs,
- enterprise incident management,
- or autonomous response actions.

This restriction is important because it allows the research to isolate the interaction between web-log content, Prompt Injection, retrieval, evidence control, and LLM reasoning.

1.6 Research Contributions

The expected contributions of this thesis are:

1. A reproducible architecture for RAG-assisted web application log analysis.
2. A controlled dataset containing paired clean and adversarial log examples.
3. An operational Prompt Injection taxonomy tailored to attacker-controlled web-log fields.
4. An experimental comparison between LLM-only, RAG, and defended RAG configurations.
5. An evaluation methodology combining conventional classification metrics with Prompt Injection robustness and evidence-grounding metrics.
6. An analysis of the effectiveness and limitations of lightweight mitigation strategies.

1.7 Thesis Structure

The remainder of this thesis is organized as follows.

Chapter 2 presents the theoretical background and related work concerning RAG, Prompt Injection, LLM applications in cybersecurity, web application log analysis, evidence grounding, and Prompt Injection defenses. The chapter concludes by identifying the research gap addressed by this work.

Chapter 3 defines the research methodology, scope, threat model, dataset strategy, experimental configurations, and evaluation metrics.

Chapter 4 presents the design of the proposed system, including preprocessing, prompt-injection detection, retrieval, the knowledge base, evidence control, secure prompt construction, and structured output.

Chapter 5 describes the dataset construction and system implementation.

Chapter 6 presents the experimental results.

Chapter 7 discusses the results, failure cases, limitations, and threats to validity.

Chapter 8 concludes the thesis and proposes directions for future work.